

Advanced Lesson 4 - Exercises: Graphics Mikey Ferguson MATH 7013 - Advanced Mathematical Modeling

```
Clear[a, b, f, g, x, y, u, v, someData];
```

1: ParametricPlot with MeshShading, from the Documentation Center

Parts 1 to 3 are a walk through the Documentation Center page for ParametricPlot. Parts 4 and 5 are below: a MeshShading example copied out of that page, and then the same example with the parameter interval moved from $0 \leq u \leq 4\pi$ to $2\pi \leq u \leq 8\pi$.

MeshShading takes a list of colours (or None) and paints the successive mesh segments of the curve with them in rotation, so the curve is drawn as an alternating dashed ribbon rather than one uniform line.

```
ParametricPlot[{u Cos[u], u Sin[u]}, {u, 0, 4 Pi}, Mesh -> 20, MeshShading -> {Red, Blue}]
```

```
ParametricPlot[{u Cos[u], u Sin[u]}, {u, 2 Pi, 8 Pi}, Mesh -> 20, MeshShading -> {Red, Blue}]
```

2: Four trigonometric functions on one graph

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2 Pi}]
```

3: Restricting the vertical range

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2 Pi}, PlotRange -> {-3, 3}]
```

4: Giving each curve its own thickness

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange → {-3, 3}, PlotStyle →
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]}]
```

5: Labelling the plot

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange → {-3, 3}, PlotStyle →
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
PlotLabel → "sin(x), cos(x), tan(x), cot(x)"]
```

6: The label in 12-point Arial Bold

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange → {-3, 3}, PlotStyle →
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
PlotLabel → Style["sin(x), cos(x), tan(x), cot(x)",
FontFamily → "Arial", FontWeight → Bold, FontSize → 12]]
```

7: Adding a frame - the finished graph

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange → {-3, 3}, PlotStyle →
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
PlotLabel → Style["sin(x), cos(x), tan(x), cot(x)",
FontFamily → "Arial", FontWeight → Bold, FontSize → 12], Frame → True]
```

8: Plotting data against a fitted curve

```
SeedRandom[20250825];
someData = Table[{0.1` i, (0.1` i)2 + 0.5` (RandomReal[] - 0.5`)}, {i, 1, 20}]
plotData = ListPlot[someData, PlotStyle → PointSize[0.02`]]
Clear[f];
f[x_] = x2;
plotF = Plot[f[x], {x, 0, 2}, PlotStyle → Thickness[0.01`]]
Show[plotData, plotF]
```

9: A least-squares fit using the basis $\{1, E^x\}$

```
Clear[g, x];
g = Fit[someData, {1, e^x}, x]
```

10: Plotting the fit, dashed

```
plotG = Plot[g, {x, 0, 2}, PlotStyle → {Thickness[0.009`], Dashing[{0.02`, 0.02`}]]]
```

11: All three on one graph

```
Show[plotData, plotF, plotG]
```

12: A parametric rose

```
ParametricPlot[{Cos[7 t] Cos[11 t], Cos[7 t] Sin[11 t]},
{t, 0, 2 π}, PlotStyle → Thickness[0.01`], AspectRatio → 1]
```

13: $\sin[x/\sin[y]]$ - the same surface eight ways

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}]
```

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 20]
```

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 50, Mesh → False]
```

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 200, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 20, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 50, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 200, Mesh → False]
```

14: Manipulate - dynamic 2-D and 3-D plots

The three cells below are Manipulate expressions. They build correctly under a plain kernel but the sliders are only live inside a Wolfram front end (Mathematica or Wolfram Player). Evaluated through Wolfram Engine under Jupyter they render as a static snapshot at the initial parameter values.

```
Manipulate[Plot[E^(-a x), {x, 0, 1}], {a, 0, 10}]

Manipulate[Plot[E^(-a x), {x, 0, 1}, PlotRange -> All], {a, 0, 10}]

Manipulate[
  Plot3D[E^(-a x - b y), {x, 0, 1}, {y, 0, 1}, PlotRange -> All],
  {a, 0, 10}, {b, 0, 10, 2}]
```

15: A dynamic algebraic calculator

```
Manipulate[Expand[(x + y)^n], {n, 0, 10, 1}]
```

16: Analysing $2x^2 + 3x^3 y == 4x^4 + 5y + 6$

```
Clear[f, x, y];
sol16 = Solve[2 x^2 + 3 x^3 y == 4 x^4 + 5 y + 6, y]
f[x_] = y /. First[sol16]
Plot[f[x], {x, -4, 4}]
Simplify[f'[x]]
Plot[f'[x], {x, -4, 4}]
Solve[f'[x] == 0, x]
NSolve[f'[x] == 0, x]
roots16 = Chop[x /. NSolve[f'[x] == 0, x]]
realRoots16 = Select[roots16, Im[#1] == 0 &]
TableForm[Table[{r, Chop[f'[r]], Chop[f''[r]],
  If[f''[r] < 0, "local maximum", "local minimum"]}, {r, realRoots16}],
  TableHeadings -> {None, {"x", "f'[x]", "f''[x]", "classification"}}]
```